

NEXUS

As-Built Architecture — Repository-Derived

Repository: DeadLock-main

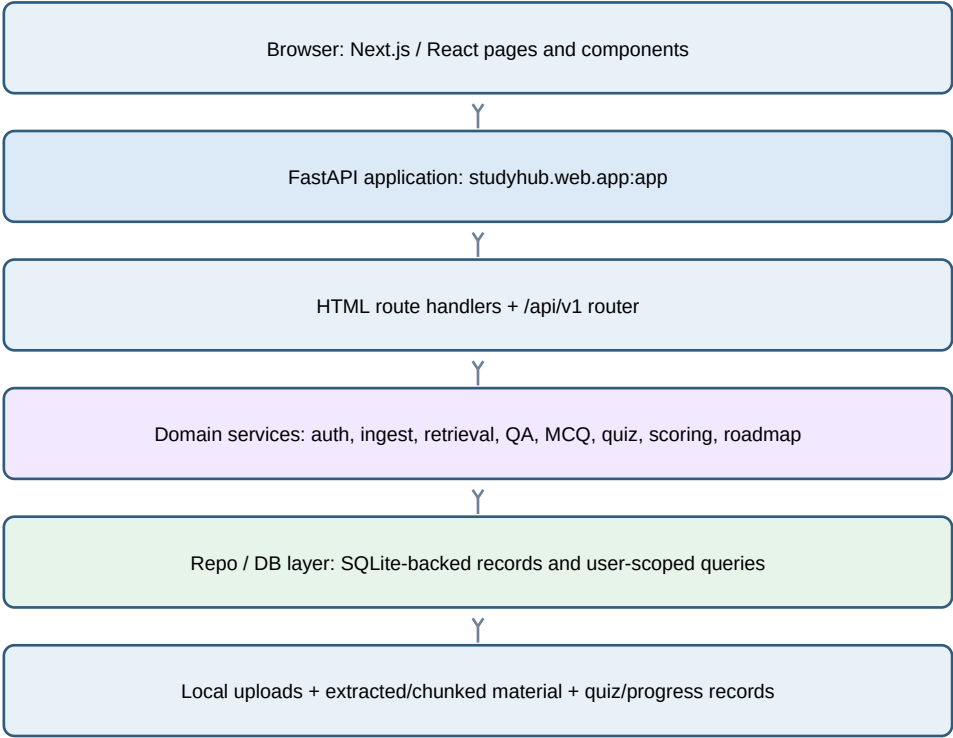
Basis: Uploaded repository ZIP, source files, route declarations, and repository documentation

This document describes the architecture found in the supplied repository. It distinguishes implemented behavior from design intent and identifies areas where the current implementation should not be described more strongly than the source supports.

Architecture scope	Evidence basis
Frontend, FastAPI web/API layers, SQLite repository, ingestion, retrieval, model routing, quiz flow, scoring, roadmap, and supporting slice library	Source inspection of studyhub/, slice/, frontend/, scripts/, tests/, and docs/ARCHITECTURE.md

1. Verified System Composition

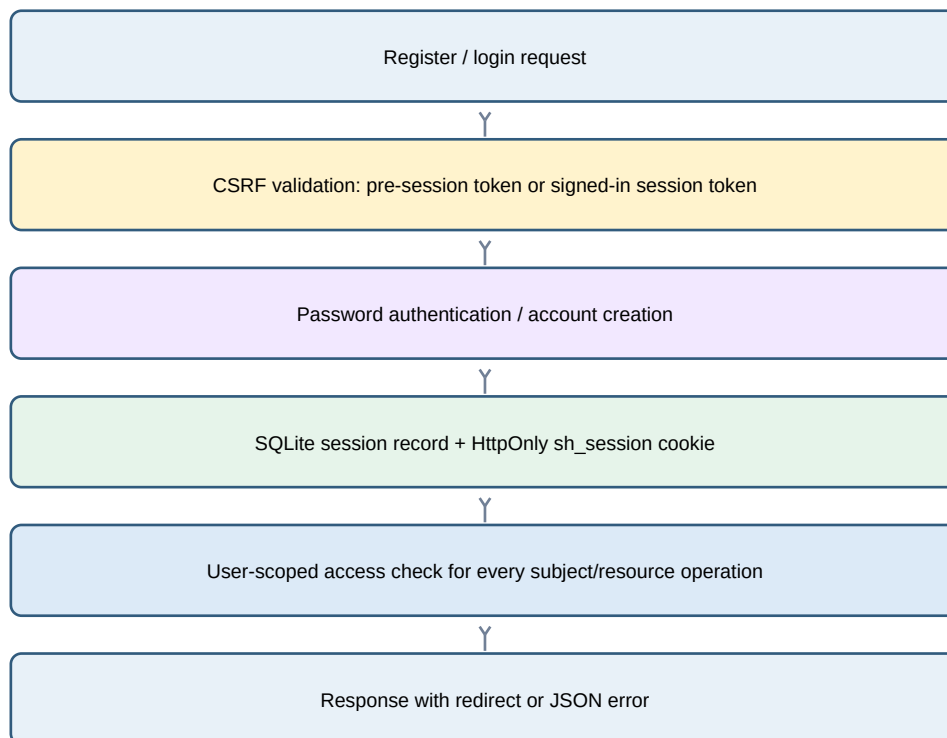
The repository contains two web presentation paths: a Next.js/React frontend under **frontend/** and a FastAPI application under **studyhub/web/**. The FastAPI application exposes both HTML routes and a versioned JSON API router. The backend uses SQLite through repository/data-access modules.



Repository evidence

Area	Observed implementation	Source location
Frontend	Next.js 15, React 19, routes for subjects, materials, ask, practice, quiz, progress, roadmap, report, account	frontend/app; frontend/package.json
Web application	FastAPI app, lifespan cleanup, security headers, session handling, HTML handlers	studyhub/web/app.py
JSON API	Versioned /api/v1 routes for session, subjects, materials, questions, MCQ, quiz, progress, notes, roadmap and account	studyhub/web/api.py
Persistence	Repo methods enforce user ownership in queries; SQLite is opened per request	studyhub/repo.py; studyhub/db.py
AI/model layer	OpenRouter and Ollama providers selected through task-aware tier construction	studyhub/models.py; slice/providers.py

2. Authentication, Session and Security Flow



Observed security controls

The FastAPI application documents and implements CSRF checks for state-changing requests, user-scoped SQL access, security response headers, upload-size checks, and server-side session cookies. The API client stores the CSRF token in memory and sends it on mutations.

Control	Repository evidence
CSRF	studyhub/web/app.py checks tokens; frontend/lib/api.js sends X-CSRF-Token for mutations
Session	auth.create_session/get_session/delete_session; HttpOnly sh_session cookie
Ownership	Repo methods receive user_id and include ownership conditions
Headers	CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Cache-Control
Upload limit	Request-level content-length check and ingest-level byte limit

Important distinction

The repository contains both the Next.js frontend and server-rendered FastAPI HTML routes. The architecture should show them as separate presentation paths sharing the backend/domain layer, rather than implying that the Next.js pages are the only interface.

3. Material Ingestion and Retrieval Architecture



Implemented ingestion behavior

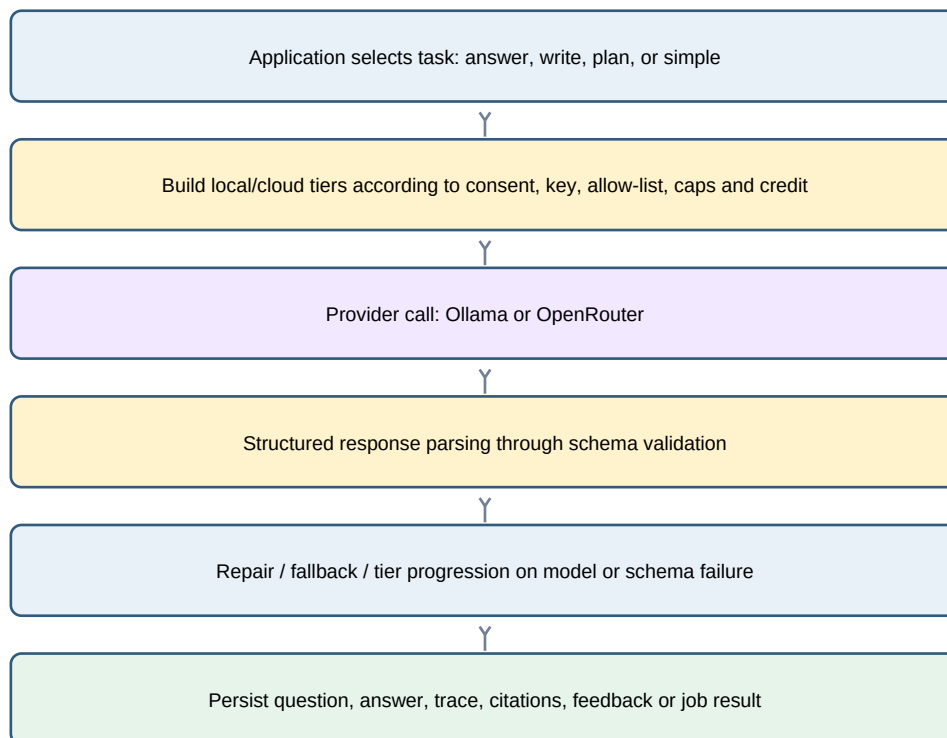
The source explicitly rejects empty bytes with “**The file is empty.**”, rejects files over the configured limit, detects duplicates using SHA-256, extracts recognized files, chunks parsed content, and records failed recognized content with a visible failed status. Suspicious passages are flagged and excluded from model-facing retrieval when the answers flag is enabled.

Implemented retrieval behavior

The retrieval module uses subject-scoped keyword processing, light stemming, SQLite FTS5/BM25 ranking, matched-term coverage, and an optional relevant-only filter. It can exclude quarantined passages from content supplied to a model. The source states that retrieval is not vector search in the StudyHub path.

Module	Responsibility
studyhub/ingest.py	Upload checks, extraction orchestration, chunking, quarantine, file storage, deletion
studyhub/extract.py / chunker.py	File extraction and chunk construction
studyhub/retrieval.py	Subject-scoped search, BM25 ordering, relevance and focus terms
studyhub/repo.py	Document, chunk, topic, and ownership persistence/access

4. AI Provider and Question-Answering Flow



Provider routing facts found in the repository

The model layer defines local Ollama and cloud OpenRouter tiers. Cloud use is gated by API-key availability, user cloud consent, model allow-list, user/app daily token caps, and available OpenRouter credit. Task-specific ordering is configured, with simple checks designated for local-first routing. The code limits cloud tiers per task and caps cloud output tokens.

Question answering

The Q&A; module searches subject material, builds prompts from passages, runs model tiers, supports revisions, and stores an outcome. The schema layer includes claims, citations, sources, verification and step fields. The repository also contains trace and feedback routes.

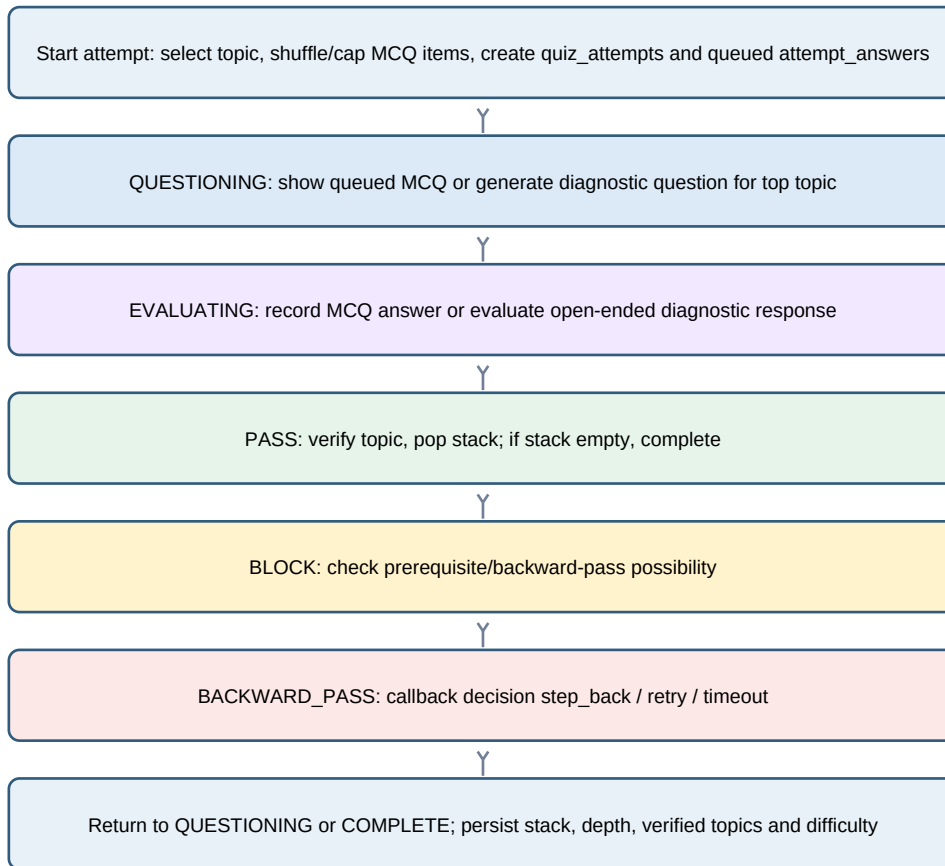
MCQ generation

The MCQ module defines typed draft/batch/pick structures, generates questions from passages, checks similarity and answer quality, supports revisions and an optional solver/independent check, and runs through background job handlers in the web application. Jobs expose status, requested/produced/rejected counts, model/tier information and steps.

Source	Verified responsibility
studyhub/models.py	Task-aware local/cloud tier construction and cloud gating
slice/providers.py	Provider protocol plus OpenRouter, Ollama and fixture providers
slice/llm.py	Model call, parsing, repair, error classes and optional spans
studyhub/qa.py	Grounded Q&A;, revisions, outcome and trace flow
studyhub/mcq.py	MCQ generation, validation, revision and job execution

5. Implemented Adaptive Quiz State Machine

The repository explicitly defines a four-state quiz flow: QUESTIONING → EVALUATING → BACKWARD_PASS → COMPLETE. The state is persisted in the quiz_attempts table between HTTP requests.



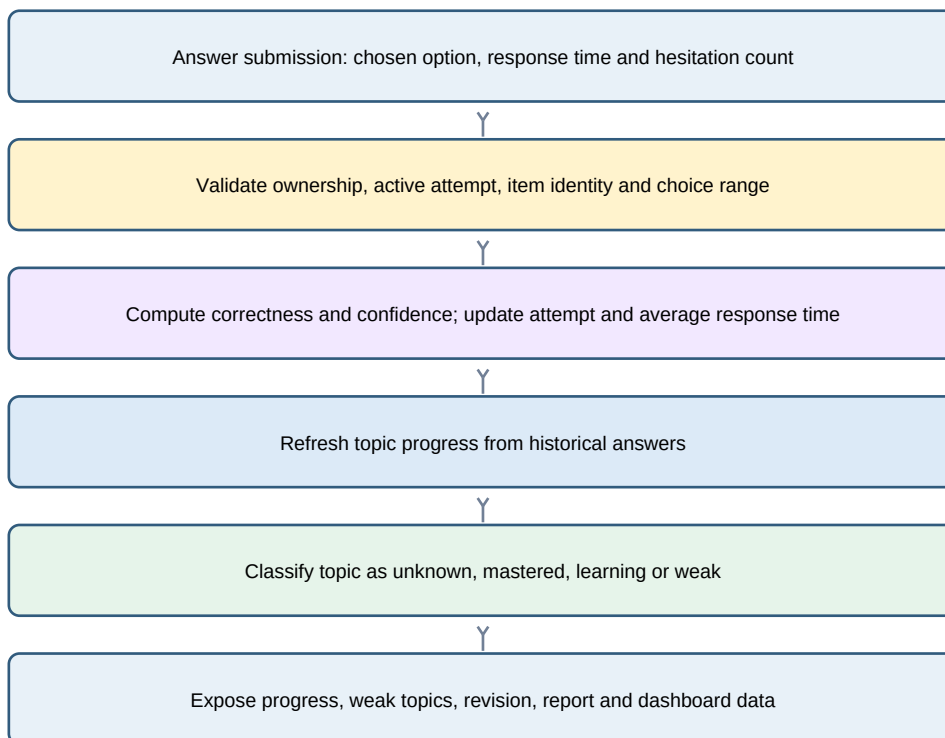
Current implementation behavior that must be represented accurately

The source comments state that **current_difficulty is 1–3 and increases on RETRY for the same topic**. In the BLOCK path, when backward stepping is not possible, the code uses **min(diff + 1, 3)**. Therefore, the current as-built architecture must show difficulty increasing in that branch; it should not replace this with a different policy unless the code is changed.

The current implementation also uses a prerequisite graph, a topic stack, a depth counter, a verified-topic list, callback state, and a 60-second timeout constant for the callback page. The web layer calls one step at a time, avoiding blocking input calls.

State field	Observed purpose
topic_stack_json	Current topic path; top of stack is current
depth	Number of backward-pass steps in the session
topics_verified_json	Topics that received PASS
callback_state	none / waiting / step_back / retry / timeout
current_difficulty	Difficulty 1–3; increased on retry in the current BLOCK branch
is_active	1 while in progress; 0 after completion

6. Scoring, Mastery, Proctoring and Progress



Verified scoring rules

The scoring module documents and implements a rule-based mastery classification: at least two answered items are required; at or above 70% correct is **mastered**; below 40% is **weak**; otherwise the state is **learning**. With fewer than two answered items, the state is **unknown**.

Confidence is calculated from response time and hesitation count using fixed thresholds. The scoring module states that it does not call an LLM.

Proctoring

The browser sends recognized event types including `tab_switch`, `full_screen_exit`, `copy_attempt`, `paste_attempt`, `browser_resize` and `auto_submit`. The backend validates event type, stores the event, updates counters where mapped, and deducts severity from `behavior_score` with a floor of zero. The result page reads a proctoring summary.

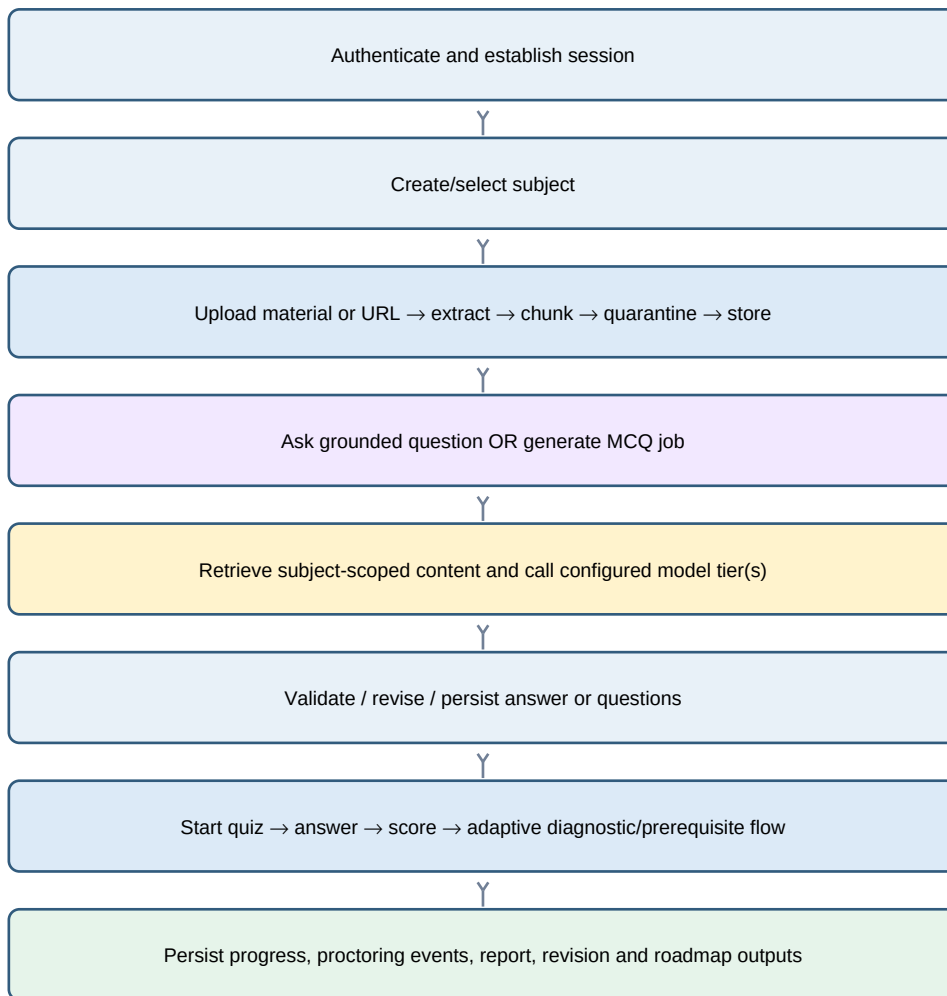
Output	Repository source
Progress and weak topics	studyhub/scoring.py; studyhub/repo.py; API progress routes
Quiz result	Answer breakdown, skipped/ended reason, diagnosis, focus events, weak topics
Report	Overall ability fields, topic rows, ability trend, attempts and backtracking
Dashboard	Totals, activity, quiz trend, topic states, outcomes, feedback and weak topics

7. Roadmap, Notes and Supporting Features

The repository includes roadmap, notes, flashcards, saved questions, search, account export/deletion, email verification and weekly digest functionality. These are real repository areas and should be included in a complete system diagram only if the presentation scope requires them.

Feature	Observed implementation
Roadmap	Skill-gap calculation, topic ordering, plan building, profile storage and coach prompt/cleanup
Notes	List/get/create/update/delete notes, including notes generated from answers
Flashcards	Flashcard listing and review endpoint
Search	Subject-scoped search and global search routes
Account	Password, export, delete, cloud consent, email verification and weekly digest routes
Background work	ThreadPoolExecutor-based submission for question, MCQ and coach processing paths

8. Repository-Based End-to-End User Flow



9. Accuracy Notes and Known Boundaries

The following points are important when presenting this architecture:

- The repository includes a separate **slice** library with append-only run storage, typed records, bounded budgets, callbacks and a generic runner. The StudyHub web flow also has its own SQLite-backed quiz state machine. These should not be merged into one identical runtime flow without tracing their actual call paths.
- The StudyHub retrieval implementation is keyword/FTS5/BM25-based. Do not label the StudyHub path as a vector database or embedding-based RAG system unless another verified code path is included.
- The source explicitly contains local Ollama and cloud OpenRouter provider paths, but actual provider usage depends on configuration, consent, API availability and task selection.
- The quiz implementation currently increases difficulty in the no-backward-pass BLOCK branch. This is a documented as-built behavior and may be a design issue, but it must not be silently changed in an architecture diagram.
- The repository contains both modern Next.js pages and FastAPI server-rendered routes. The final presentation should make the two interfaces and their shared backend relationship clear.
- A repository-derived diagram demonstrates code structure, not proof that every user-facing feature has passed live testing. Test evidence should be linked separately.

10. Source Map for Review

Architecture area	Primary files
Web/API entry points	studyhub/web/app.py; studyhub/web/api.py
Data and ownership	studyhub/repo.py; studyhub/db.py; studyhub/auth.py
Ingestion and retrieval	studyhub/ingest.py; extract.py; chunker.py; retrieval.py
AI/model routing	studyhub/models.py; slice/providers.py; slice/llm.py
Q&A; and MCQ	studyhub/qa.py; studyhub/mcq.py
Adaptive quiz	studyhub/quiz_flow.py; quiz_agents.py
Scoring/proctoring	studyhub/scoring.py
Roadmap and analytics	studyhub/roadmap.py; insights.py; report API routes
Frontend	frontend/app; frontend/components/nexus; frontend/lib
Testing	tests/; frontend/e2e/; docs/qa-evidence/

Conclusion

This document is an as-built architecture description derived from the supplied repository, not a proposed future design. It captures the implemented application layers, material pipeline, model routing, quiz state machine, scoring, proctoring, and supporting features while retaining important implementation boundaries and current behaviors.